ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837

Παράλληλα Συστήματα - Εργασία 3

Ιωάννης Γρηπιώτης (sdi2100028)
Θεμιστοκλής Παπαθεοφάνους (sdi2100227)

ΠΕΡΙΛΗΨΗ

Όλες οι ασκήσεις εκτελέστηκαν πάνω στο σύστημα linux02.di.uoa.gr του Πανεπιστημίου Αθηνών, το οποίο διαθέτει:

- επεξεργαστή Intel Core i7-6700 4c/8t @ 3.40GHz,
- 16GB μνήμη RAM,
- λειτουργικό σύστημα Ubuntu 20.04.6 LTS,
- έκδοση πυρήνα 5.4.0-216-generic,
- έκδοση μεταγλωττιστή GCC 9.4.0

Κάθε άσκηση εκτελέστηκε 10 φορές με την μέση τιμή του χρόνου εκτέλεσης να υπολογίζεται από το ίδιο το εκτελέσιμο.

Κάθε άσκηση εκτελείται από το εκτελέσιμο ως εξής:

```
./scripts/run_<exercise_name>.sh
```

όπου το `<exercise_number>` είναι ο αριθμός της άσκησης που πρόκειται να εκτελεστεί και τα `FLAGS` είναι οι παράμετροι (οι οποίες έχουν το πρόθημα `-f`) που δίνονται στην εκάστοτε άσκηση.

ΑΣΚΗΣΗ 1.1

Περιγραφή Υλοποίησης

Το πρόγραμμα υλοποιεί τον παράλληλο πολλαπλασιασμό δύο πολυωνύμων βαθμού n χρησιμοποιώντας MPI. Η εκτέλεση χωρίζεται στις ακόλουθες φάσεις (εξαιρώντας τη δημιουργία και αρχικοποίηση των πολυωνύμων):

- **Send**: Διανομή δεδομένων μέσω `MPI_Bcast` (πολυώνυμο b) και `MPI_Scatterv` (τμήματα του πολυωνύμου a)
- **Compute**: Υπολογισμός τοπικού γινομένου σε κάθε process
- **Receive**: Συλλογή και άθροιση αποτελεσμάτων μέσω `MPI_Reduce`

Ανάλυση Επιδόσεων MPI

Ο Table 1 παρουσιάζει τους μέσους χρόνους εκτέλεσης ανά φάση για μέγεθος πολυωνύμου $n = 100000$.

Proc	Send (ms)	Compute (s)	Recv (ms)	Total (s)	Speedup
1	0.8	30.42	0.4	30.42	1.00
2	0.7	15.60	1.1	15.61	1.95
4	1.6	8.22	1.4	8.24	3.69
8	2.9	4.11	14.7	4.14	7.34
16	5.3	2.05	22.2	2.13	14.31
32	8.9	1.03	23.8	1.08	28.08
40	14.9	0.82	29.9	0.88	34.66

Table 1: Χρόνοι εκτέλεσης MPI ανά φάση (size=100,000)

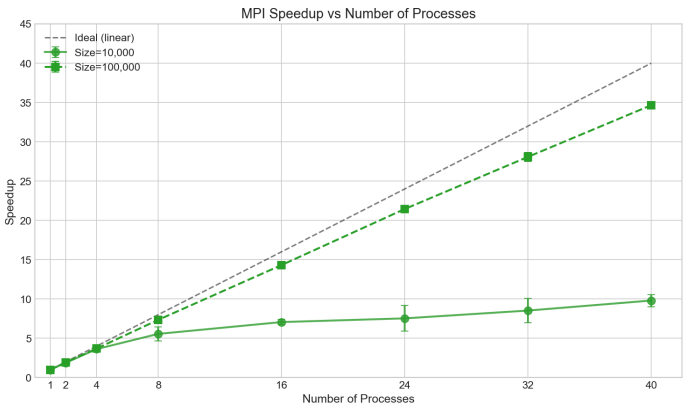


Figure 1: MPI Speedup συναρτήσει αριθμού processes

Παρατηρούμε ότι το MPI επιτυγχάνει σχεδόν γραμμικό speedup για μέγεθος 100,000, με efficiency 87% στις 40 διεργασίες. Για μικρότερα μεγέθη (10,000) το speedup περιορίζεται λόγω του μεγαλύτερου σχετικού overhead επικοινωνίας. Το communication overhead παραμένει χαμηλό (45ms συνολικά) σε σχέση με τον χρόνο υπολογισμού, όπως φαίνεται και στο Figure 2.

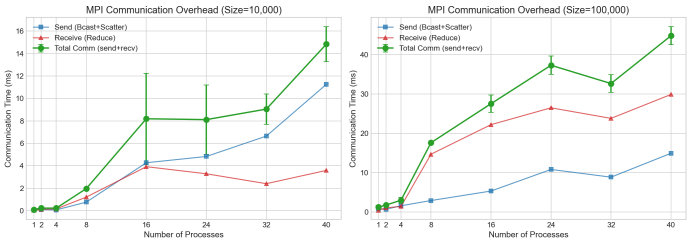


Figure 2: MPI Communication Overhead ανά μέγεθος

Σύγκριση MPI vs Pthreads vs OpenMP

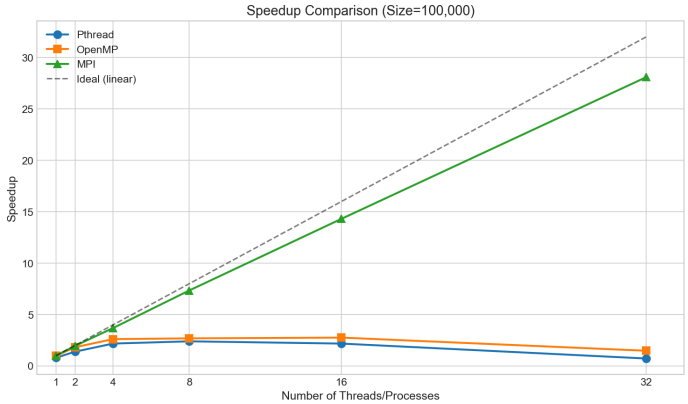


Figure 3: Σύγκριση Speedup μεταξύ MPI, Pthreads και OpenMP

Το MPI υπερτερεί σημαντικά έναντι των Pthreads και OpenMP:

- **MPI**: Speedup 35x με 40 processes, σχεδόν γραμμική κλιμάκωση
- **OpenMP**: Μέγιστο speedup 2.8x (8-16 threads), σταδιακή επιδείνωση
- **Pthreads**: Μέγιστο speedup 2.5x (4-8 threads), σημαντική επιδείνωση στα 32 threads (speedup < 1)

Συμπεράσματα

Η διαφορά επιδόσεων οφείλεται στα εξής:

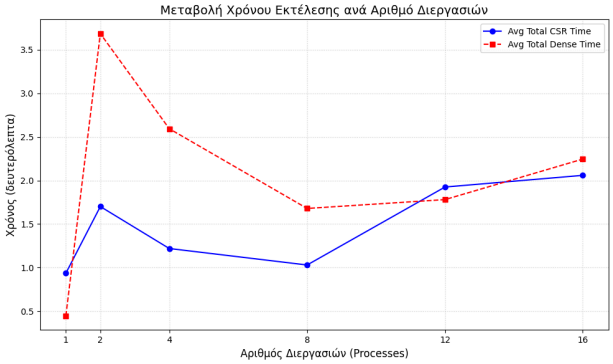
1. **Distributed vs Shared Memory**: Το MPI εκμεταλλεύεται πολλαπλούς κόμβους με ξεχωριστή μνήμη, αποφεύγοντας το memory bandwidth bottleneck που περιορίζει τα Pthreads/OpenMP σε έναν κόμβο.
2. **Scalability**: Το MPI κλιμακώνει πέρα από τους πυρήνες ενός μηχανήματος, ενώ τα Pthreads/OpenMP περιορίζονται από το hardware του μοναδικού κόμβου (4 cores / 8 threads στο linux02).

ΑΣΚΗΣΗ 1.2

Αρχικά έγιναν κάποια πειράματα προκειμένου να ελεγχθεί το πόσο καλά μπορεί να κάνει **scaling** η εφαρμογή. **Scaling** θεωρείται το γεγονός στο οποίο όταν μεταβάλλεται με σταθερό ρυθμό ο χρόνος, καθώς αυξάνεται ο αριθμός των **processes**, έχοντας μια λογική μεταβολή (σχετικά γρήγορη). Στα παρακάτω πειράματα κρατήθηκε σταθερό το **μέγεθος**, δίνοντας το μεγαλύτερο δυνατό 10^4, **αριθμός επαναλήψεων** γινόμενο πίνακα-διανύσματος 10. Παρατηρήθηκε η συμπεριφορά τόσο με 90% **Sparsity** όσο και με 70%.

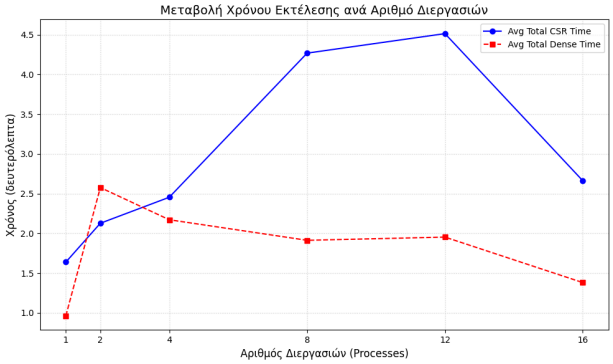
Scaling

Αποτελέσματα με 90% **Sparsity**.



Processes	CSR (Sup)	CSR (Eff)	Dense (Sup)	Dense (Eff)
1	x1	100%	1.000	100%
2	x0.550	27.5%	x0.122	6.1%
4	x0.767	19.2%	x0.173	4.3%
8	x0.908	11.4%	x0.267	3.3%
12	x0.486	4.1%	x0.252	2.1%
16	x0.455	2.8%	x0.199	1.2%

Αποτελέσματα με 70% **Sparsity**.



Processes	CSR (Sup)	CSR (Eff)	Dense (Sup)	Dense (Eff)
1	x1	100%	1.000	100%
2	x0.770	38.5%	x0.374	18.7%
4	x0.668	16.7%	x0.444	11.1%
8	x0.384	4.8%	x0.504	6.3%
12	x0.363	3.0%	x0.493	4.1%
16	x0.616	3.8%	x0.698	4.4%

Συμπερασματα & Παρατηρησεις

Τόσο το 90% όσο και το 70% έχουν πάρα πολύ κακό **scaling**. Δεν παρατηρείται καθόλου **Speed Up**, παρά μόνο επιβράδυνση.

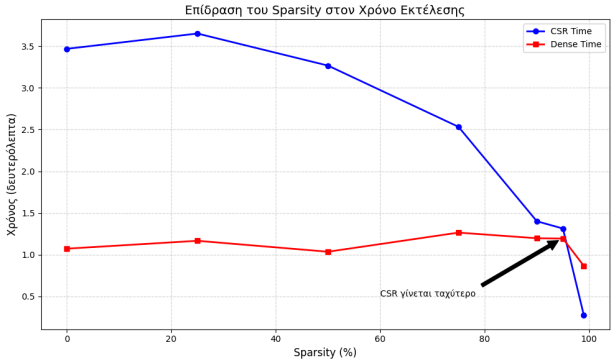
Στο 90%, ως καλύτερη τιμή εμφανίζονται τα 8 και το σειριακό για την **CSR** αναπαράσταση, έχοντας βέβαια πολύ χειρότερο **Efficiency** στα 8. Παρόμοια συμπεριφορά έχει και η **Dense**, παρουσιάζοντας πιο απότομες μεταβολές, μιας και γίνεται πάρα πολύ αργή για **2 processes**.

Στο 70% παρατηρείται κυριαρχία της **Dense** αναπαράστασης σχεδόν σε όλη τη διάρκεια του πειράματος, έχοντας μια ελαφριά τάση πτώσης. Ίδια συμπεριφορά, με μεγαλύτερη οξύτητα ως προς την πτώση, αποκτά και η **CSR** μορφή προς το τέλος του πειράματος.

Χρονος & Sparsity

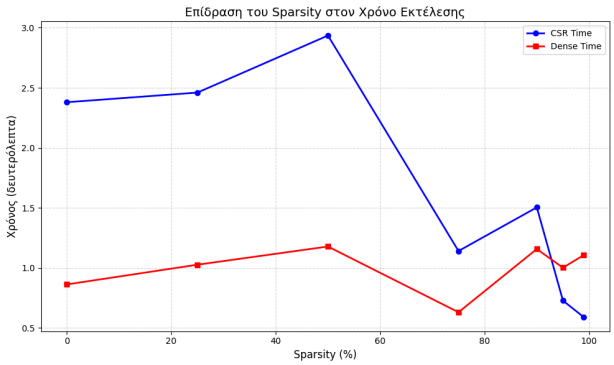
Παρακάτω έγιναν κάποια πειράματα προκειμένου να εξεταστεί η συμπεριφορά της εφαρμογής στη μεταβολή του **Sparsity**, τόσο σε διαφορετικά όσο και σε ίδια **Nodes** (στον ίδιο υπολογιστή). Για να γίνουν τα πειράματα, κρατήθηκε σταθερό το μέγεθος πίνακα στα 10^4, **αριθμός επαναλήψεων** 1 και 4 **processes**.

Αποτελέσματα για πείραμα σε **διαφορετικά nodes** σε **[sec]**



Sparsity	Total CSR	Total Dense	Diff (CSR-DENSE)
0	3.469	1.070	2.398
25	3.652	1.165	2.487
50	3.268	1.033	2.234
75	2.533	1.26	1.269
90	1.399	1.195	0.204
95	1.312	1.191	0.120
99	0.272	0.864	−0.591

Αποτελέσματα για πείραμα σε **ίδια nodes** σε **[sec]**



Sparsity	Total CSR	Total Dense	Diff (CSR-DENSE)
0	2.379	0.862	1.516
25	2.460	1.027	1.432
50	2.934	1.178	1.756
75	1.141	0.631	0.509
90	1.504	1.157	0.346
95	0.728	1.002	−0.273
99	0.590	1.107	−0.517

Συμπερασματα & Παρατηρησεις

Παρατηρείται ότι τόσο στο πείραμα με τα **ίδια nodes** όσο και σε αυτό με **διαφορετικά** υπάρχει η ίδια συμπεριφορά, τόσο για την **CSR** αναπαράσταση,

η οποία εμφανίζει κάθοδο, όσο και για τη **Dense**, η οποία παραμένει σχετικά σταθερή. Η διαφορά είναι ότι στο πείραμα με τα **διαφορετικά nodes** οι μεταβολές είναι πολύ πιο ήπιες και αρμονικές, σε αντίθεση με το πείραμα στα **ίδια nodes**, στο οποίο είναι απότομες.

Όσον αφορά τη συμπεριφορά των διαγραμμάτων, αυτό είναι εντελώς αναμενόμενο, μιας και η «δύναμη» της **CSR** αναπαράστασης έγκειται στο γεγονός ότι υπάρχει αραιός πίνακας. Όσο πιο αραιός είναι, τόσο λιγότερες είναι οι πράξεις που θα γίνουν.

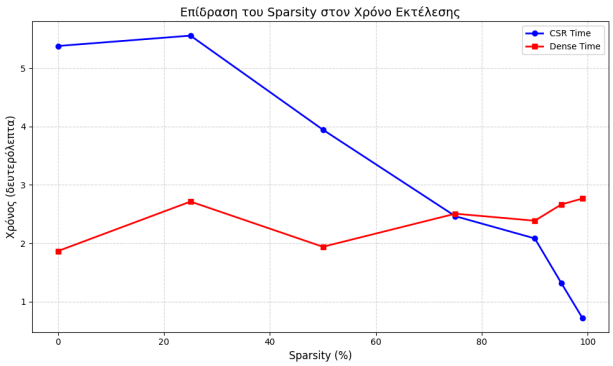
Τέλος, κάποιος μπορεί να παρατηρήσει ότι οι χρόνοι των **Dense** αναπαραστάσεων, τόσο στο πείραμα με **πολλά nodes** όσο και σε αυτό με το 1, είναι αρκετά παρόμοιοι, σε αντίθεση με τους χρόνους **CSR** των 2 πειραμάτων. Αυτό γίνεται εξαιτίας ενός **trade-off** που παρουσιάζεται και έχει να κάνει με τον καταμερισμό **εργασίας** σε **πολλά nodes**. Άμα υπάρχουν πολλά **nodes**, τότε κάθε διεργασία είναι πιο **ελεύθερη** στην όσο το δυνατόν καλύτερη αξιοποίηση της **CPU**. Αντίθετα, άμα είναι όλα σε 1 **node**, υπάρχει ανταγωνισμός για το ποιος θα επικρατήσει στην **CPU**. Κάτι που δυσχεραίνει ακόμη αυτήν την κατάσταση είναι το γεγονός ότι υπάρχουν και άλλοι συνάδελφοι που μπορεί να απασχολούν τους ίδιους πόρους, προσδίδοντας έτσι **θόρυβο** στα πειράματα.

Ακόμη, η επικοινωνία πάνω από το δίκτυο με ίδιο σταθμό και αφετηρία είναι πολύ πιο γρήγορη απ' ό,τι σε διαφορετικούς, περιμένοντας έτσι κάποιος ίσως διαφορετικό αποτέλεσμα από το συγκεκριμένο.

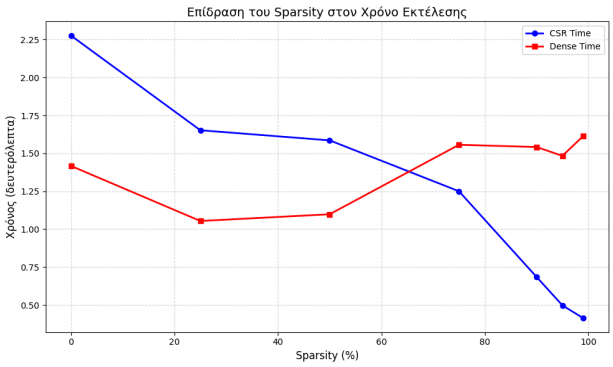
Περιεργεια

Λόγω περιέργειας έγιναν ακόμη κάποια πειράματα, κρατώντας **σταθερό το μέγεθος** πίνακα στα 10^4, **αριθμός επαναλήψεων** 10, αντί για 1 που ήταν πριν, και 4 **processes**. Ο σκοπός ήταν να μελετηθεί η αντίδραση της παράλληλης εφαρμογής καθώς αυξάνεται το **Sparsity**, με 10 επαναλήψεις τόσο σε **διαφορετικά nodes** όσο και στο **ίδιο node**.

Αποτελέσματα για πείραμα σε **διαφορετικά nodes** σε **[sec]**

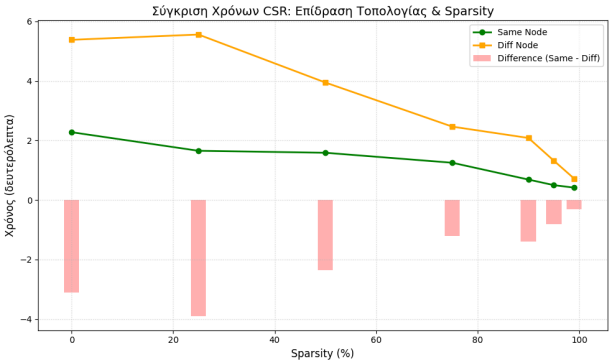


Αποτελέσματα για πείραμα σε **ίδια nodes** σε **[sec]**



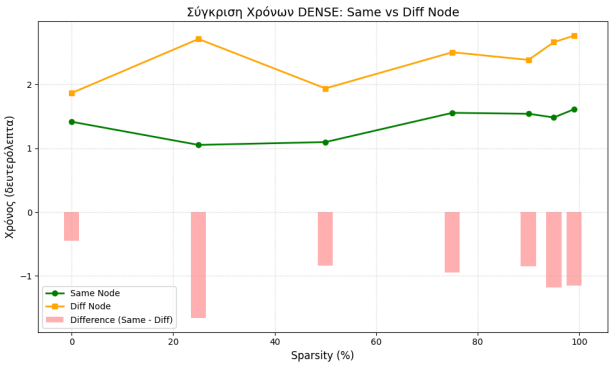
Τα παραπάνω αποτελέσματα έχουν παρόμοια συμπεριφορά με αυτά της 1 **επανάληψης**. Κάτι, όμως, που έχει ιδιαίτερο ενδιαφέρον δεν είναι η αυτούσια συγκεκριμένη σύγκρισή τους, αλλά η διαφορά χρόνων που παρουσιάζει τόσο η αντίστοιχη **CSR** όσο και η αντίστοιχη **Dense** αναπαράσταση πάνω σε **διαφορετικά** και **ίδια nodes**.

Αποτελέσματα **CSR** αναπαράστασης σε **ίδια** και **διαφορετικά nodes σε [sec]**



Sparsity	Same Nodes	Diff Nodes	Diff (Same-Diff)
0	2.276	5.380	−3.104
25	1.652	5.558	−3.905
50	1.585	3.945	−2.360
75	1.250	2.463	−1.213
90	0.684	2.082	−1.397
95	0.497	1.318	−0.821
99	0.413	0.717	−0.304

Αποτελέσματα **Dense** αναπαράστασης σε **ίδια** και **διαφορετικά nodes** σε **[sec]**



Sparsity	Same Nodes	Diff Nodes	Diff (Same-Diff)
0	1.416	1.866	−0.450
25	1.053	2.714	−1.660
50	1.098	1.939	−0.841
75	1.556	2.506	−0.949
90	1.541	2.386	−0.844
95	1.483	2.664	−1.180
99	1.613	2.766	−1.153

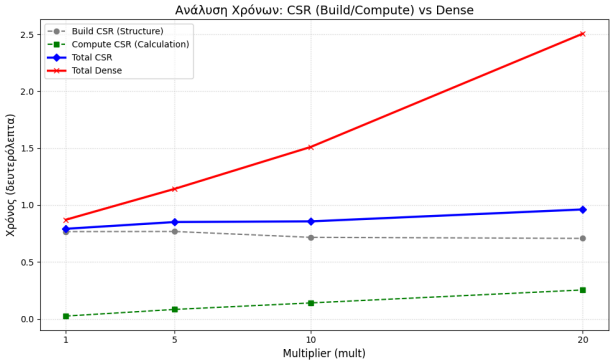
Συμπερασματα & Παρατηρησεις

Όπως μπορεί να παρατηρήσει κανείς, τα συγκεκριμένα αποτελέσματα είναι αρκετά ενδιαφέροντα, ειδικά αν κάποιος λάβει υπόψη του τα 2 προηγούμενα, που αφορούν ακριβώς το ίδιο πείραμα, αλλάζοντας μόνο τον **αριθμό των επαναλήψεων** του γινομένου **Matrix-Vector**. Παρατηρείται παρόμοια συμπεριφορά όσον αφορά τις τιμές στα πειράματα, τόσο για την **CSR** όσο και για την **Dense** αναπαράσταση. Ουσιαστικά, λόγω των περισσότερων επαναλήψεων σε αντίθεση με πριν, φάνηκε η καλύτερη επικοινωνία δεδομένων

πάνω από το ίδιο δίκτυο, **ίδιο node**. Στο συγκεκριμένο πείραμα έπρεπε να γίνει συγχρονισμός δεδομένων για το **vector** 10 φορές, κάνοντας έτσι αρκετά κοστοβόρα την υλοποίηση· παρ' όλα αυτά, όμως, η γρήγορη μεταφορά των δεδομένων μπορεί να κλιμακώσει πολύ εύκολα και γρήγορα τη διαφορά χρόνου.

Τελευταιο πειραμα

Παρακάτω παρουσιάζονται αποτελέσματα σε **[sec]**, προκειμένου να παρατηρηθεί η μεταβολή του χρόνου **κατασκευής** και **υπολογισμού** της **CSR** μορφής. Πιο συγκεκριμένα, έγινε μεταβολή του **αριθμού του γινομένου**, κρατώντας σταθερό το μέγεθος του **πίνακα** 10^4, **Sparsity** 90% και 4 **processes**.



Mult	Build CSR	Compute CSR	Total CSR	Total Dense
1	0.766	0.024	0.791	0.871
5	0.767	0.083	0.850	1.142
10	0.716	0.140	0.856	1.510
20	0.707	0.254	0.961	2.505

Παρατηρείται ότι ο χρόνος κατασκευής παραμένει σχετικά σταθερός και μεταβάλλεται λόγω θορύβου, ενώ ο υπολογισμός του γινομένου αυξάνεται όσο μεγαλώνουν τα **loops**, κάτι που είναι αναμενόμενο. Λόγω του ότι το πείραμα είναι σε πολύ μεγάλο **Sparsity** (90%), το αποτέλεσμα είναι αναμενόμενο και δεν κινεί το ενδιαφέρον για περαιτέρω μελέτη.

Γενικη Παρατηρηση

Ένας πιθανός λόγος που δεν υπάρχει **Speed Up** ή που κάποια από τα αποτελέσματα είναι πολύ «χειρότερα» απ' ό,τι θα περίμενε κανείς, είναι εξαιτίας της φύσης του προβλήματος και της χρήσης του μέσου.

Φυση του προβληματος

Το πρόβλημα, τόσο σε CSR όσο και σε Dense μορφή, στον πυρήνα του παραμένει ένα γινόμενο Matrix-Vector. Είναι μαθηματικά αποδεδειγμένο ότι δεν γίνεται να χωριστεί το **vector**, ακόμα και αν μπορεί να χωριστεί σε ίσα μέρη τόσο η **CSR** όσο και η **Dense** αναπαράσταση του πίνακα. Αυτό έχει ως αποτέλεσμα να πρέπει όλα τα **processes** να έχουν ολόκληρο το διάνυσμα, όσο μεγάλο και αν είναι, ακόμα και αν κατέχουν μόνο μέρος της αναπαράστασης του πίνακα. Διαφορετικά, θα προέκυπτε λανθασμένο αποτέλεσμα.

Το Μεσο

Το μέσο παραλληλοποίησης της εφαρμογής, δηλαδή η χρήση της **MPI**, συνεισφέρει σε αυτήν την αλλοίωση των χρόνων των πειραμάτων. Εξαιτίας της **MPI**, ο μόνος τρόπος που έχουν για να επικοινωνήσουν τα **processes** είναι πάνω από το δίκτυο, σε αντίθεση με την προηγούμενη προσέγγιση με τη χρήση του **OpenMP** πάνω στην ίδια μνήμη. Αυτό, αν και θεωρητικά θα έπρεπε να βελτιστοποιεί τον χρόνο, κάνει ακριβώς το αντίθετο. Τόσο το λειτουργικό σύστημα όσο και η χρήση της **MPI** για επικοινωνία πάνω από το δίκτυο καθιστούν ανέφικτη τη **βελτιστοποίηση** του χρόνου. Κάθε φορά που πρέπει να σταλούν δεδομένα, είτε στην αρχή (metadata, μεγέθη κ.λπ.) όσο και αργότερα λόγω συγχρονισμού προκειμένου να ενωθούν τα αποτελέσματα (**loops**), η εφαρμογή υφίσταται έντονο **performance bleeding**, το οποίο δεν μπορεί να γίνει **overlap** και δεν αφορά τον **CPU computation time**. Επομένως, υπάρχει ένα σημαντικό **trade-off** όσον αφορά την αύξηση των **processes**, το μέγεθος των πινάκων-διανυσμάτων και τον αριθμό των επαναλήψεων.

Γενικο Συμπερασμα — Hot-values, Switch Point

Με βάση όλα τα παραπάνω πειράματα, μπορεί κανείς να καταλήξει στο ίδιο συμπέρασμα με το αντίστοιχο πρόβλημα και την υλοποίηση με **OpenMP**. Δεν μπορεί κάποιος να αποφασίσει με βεβαιότητα εκ των προτέρων ποια από τις 2 αναπαραστάσεις είναι η καλύτερη. Αυτό οφείλεται στο γεγονός ότι η σύγκριση δεν επηρεάζεται από έναν και μοναδικό παράγοντα, αλλά από ένα σύνολο αυτών. Πιο συγκεκριμένα, επηρεάζεται από το **μέγεθος**, το **Sparsity** (ποσοστό μηδενικών), τον αριθμό των **επαναλήψεων γινόμενου**, ακόμη και τον αριθμό των **processes**, καθώς και τον **καταμερισμό** τους. Σε όλα αυτά συμβάλλει, βέβαια, και η **MPI**, η οποία επηρεάζεται από το λειτουργικό σύστημα, αλλά ακόμη και από την ποιότητα και την ταχύτητα του δικτύου, το βασικό μέσο για τη μεταφορά της πληροφορίας.

ΑΣΚΗΣΗ 1.3

Ευχαριστούμε τον xmamalou για το template της αναφοράς.

Ιωάννης Γρηπιώτης (sdi2100028)
Θεμιστοκλής Παπαθεοφάνους (sdi2100227)